

GRADUATION PROJECT

Web Security Assessment Platform

Penetration Testing & Application Security – E-FL

National Telecommunication Institute (NTI)

PROJECT FOCUS

An integrated assessment workflow combining reconnaissance, HTTP security analysis, behavioral profiling, and unified risk assessment.

TEAM

Malak Mahmoud

Jana Muhammad Ali

Mahmoud Basem Ibrahim Ismail

Ahmed Fouad

Prepared as a technical project report

01 | Executive Summary

This graduation project presents an integrated Web Security Assessment Platform developed within the Penetration Testing & Application Security – E-FL program at the National Telecommunication Institute (NTI). The platform brings multiple assessment capabilities into a single workflow: HTTP security scanning, security-header assessment, passive user discovery, OSINT lookup, robots.txt analysis, security behavior profiling, and unified risk assessment.

The engineering objective is not only to identify individual security findings, but to connect observations from different layers and provide a more meaningful view of a target’s security posture. The master integration layer coordinates the core assessment flow and combines the behavior-engine score, HTTP scanner score, and a robots.txt-related penalty into a unified score and rating.

ASSESSMENT FLOW

Target → Reconnaissance → HTTP & Security Analysis → Behavioral Analysis → Risk Correlation → Unified Assessment

02 | Project Objectives

- Build a practical, integrated workflow for authorized web-security assessment.
- Automate selected HTTP, reconnaissance, and configuration checks.
- Evaluate security-relevant HTTP headers and configuration indicators.
- Analyze security behavior across transport, redirects, cookies, CORS, HTTP methods, and information disclosure.
- Detect behavioral anomalies and correlate related security indicators.
- Produce a consolidated risk score that combines multiple assessment layers.
- Apply penetration-testing and application-security concepts in a collaborative engineering project.

03 | Platform Architecture

The repository is organized as a modular security toolkit with a Flask application layer, frontend assets, specialized assessment modules, and a master integration layer. The architecture separates collection, analysis, and orchestration responsibilities while allowing the frontend to consume consolidated results through API endpoints.

Layer	Component	Responsibility
Application	app.py	Flask application and API blueprint registration.
Integration	master_link.py	Orchestrates scanner, behavior engine, robots analysis, user discovery, and unified scoring.
HTTP Security	http_scanner.py	HTTP capture, headers, cookies, redirects, methods, and security findings.
Behavioral Analysis	security_behavior_engine.py	Behavior profiling, anomaly detection, correlation, risk classification, and recommendations.
Reconnaissance	user_discovery.py	Passive discovery of exposed emails, social handles, author metadata, and related identity indicators.
OSINT	osint.py / osint_api.py	Username, email, IP, and phone intelligence

Reconnaissance	robots_analyzer.py	lookup capabilities. robots.txt retrieval, parsing, interesting-path detection, and reporting.
Frontend	index.html / script.js / style.css	User-facing interface and presentation of assessment results.

04 | Core Security Capabilities

HTTP Security Scanner

Performs the base HTTP assessment and captures response status, final URL, headers, cookies, redirect behavior, and related observations. The scanner also supports checks around security headers, cookies/configuration, HTTP methods, redirects, and weighted findings.

Security Headers Assessment

Evaluates security-relevant HTTP headers, including Content Security Policy (CSP) and HTTP Strict Transport Security (HSTS), and records findings with severity-oriented scoring.

User Discovery

Uses passive site-side evidence: exposed emails, social handles, meta/JSON-LD author information, and selected WordPress author/user indicators. The implementation explicitly keeps this discovery step separate from manual OSINT lookups.

OSINT Lookup Layer

Provides separate lookup capabilities for usernames, emails, IP addresses, and phone numbers. The API layer exposes dedicated endpoints so an operator can investigate an identifier that is in scope for the assessment.

robots.txt Analyzer

Retrieves and parses robots.txt, extracts user-agent, disallow, allow, and sitemap directives, and detects potentially interesting paths using keyword-based categories and severity.

Security Behavior Profiling Engine

Analyzes already-collected scan data instead of performing its own network collection. The engine builds profiles, evidence observations, behavioral anomalies, correlated risks, recommendations, and an overall score.

Unified Risk Assessment

Combines successful behavior-engine and HTTP-scanner scores using weighted aggregation, then applies a bounded penalty for high-severity interesting robots.txt paths to produce a final percentage and rating.

05 | Security Behavior Profiling Engine

The Security Behavior Profiling Engine is a dedicated analytical layer designed to interpret security-related behavior rather than relying only on isolated findings. The implementation receives the network capture produced by the HTTP scanner and

analyzes it without fetching the target a second time.

05.1 Behavioral Domains

Domain	Weight	Analysis Focus
Transport Security	22%	HTTPS enforcement, cleartext exposure, redirect policy, HSTS policy, and protocol consistency.
Redirect Behavior	16%	Redirect-chain behavior, transitions, domain changes, and final transport behavior.
Cookie Behavior	20%	Cookie attributes, session-protection indicators, scope, prefixes, cross-site behavior, and consistency.
CORS Behavior	16%	Origin trust, credential exposure, preflight behavior, and related CORS policy indicators.
HTTP Method Behavior	14%	HTTP method exposure and behavior, including assessment of risky methods.
Information Disclosure	12%	Indicators associated with server or technology disclosure and related response information.

05.2 Analytical Output

- Evidence observations representing the underlying technical evidence.
- Behavioral anomalies representing deviations or suspicious patterns.
- Correlated risks that connect multiple indicators.
- Overall assessment with score and risk classification.
- Recommendations derived from the analyzed security behavior.

KEY ENGINEERING VALUE

The engine adds context and correlation: multiple observations can contribute to a higher-level interpretation of security posture instead of remaining as unrelated findings.

06 | Unified Risk Assessment

The master integration layer calculates the unified score only from modules that successfully return an assessment. The behavior-engine score contributes 60% and the combined HTTP scanner score contributes 40%. When robots.txt is available, each high-severity interesting path contributes a small penalty, capped at 10 points.

Unified Rating	Score Range
Excellent	90–100
Good	75–89
Fair	50–74
Poor	25–49

07 | Team Contributions

The project was developed collaboratively, with each member responsible for a defined technical area. The contributions below reflect the modules and responsibilities represented in the project source.

Malak Mahmoud | [Security Behavior Profiling Engine](#)

Designed and implemented the dedicated behavioral security-analysis layer. The work covers behavior profiling over collected HTTP/network observations, evidence modeling, anomaly detection, risk correlation, risk classification, overall scoring, and recommendation generation. The engine also organizes analysis across transport security, redirect behavior, cookie behavior, CORS behavior, HTTP methods, and information disclosure.

Jana Muhammad Ali | [Backend & Frontend Development](#) • [HTTP Scanner](#) • [User Discovery](#)

Developed the application-facing backend/frontend integration and contributed to the HTTP security scanning and passive user-discovery capabilities. The HTTP scanner captures target responses and evaluates headers, cookies, redirects, HTTP methods, and related configuration indicators. User discovery extracts exposed identity indicators such as emails, social handles, author metadata, and selected WordPress user signals.

Mahmoud Basem Ibrahim Ismail | [OSINT Lookup Layer](#)

Developed the OSINT lookup functionality and its API integration. The implementation supports username, email, IP, and phone intelligence workflows, including validation and enrichment such as DNS/RDAP-related IP information, email-domain security indicators, and phone-number metadata.

Ahmed Fouad | [robots.txt Analyzer](#)

Developed the robots.txt analysis component. The analyzer retrieves robots.txt, parses user-agent/disallow/allow/sitemap directives, identifies potentially interesting paths using security-relevant keywords and severity categories, and provides structured reporting/output capabilities.

08 | End-to-End Integration

The master scan begins with the HTTP scanner. When the scanner succeeds, its captured network data is passed directly to the behavior engine, avoiding a second target fetch by that engine. Robots.txt analysis and passive user discovery are then executed as separate components. The master layer aggregates the relevant outputs and exposes the consolidated result to the application layer.

DESIGN PRINCIPLE

Collection and analysis are separated: the HTTP scanner performs network collection for the shared capture, while the behavior engine focuses on analyzing that evidence.

09 | Security, Scope & Responsible Use

The platform is intended for authorized security testing, education, and controlled laboratory environments. Operators should only assess systems they own or have explicit permission to test. The OSINT endpoints are designed as separate,

manual lookups for identifiers that are already in scope, while user discovery remains focused on information the target site itself exposes.

The project repository should not contain real API keys, passwords, tokens, private keys, or other credentials. Generated Python cache files and local Git metadata should remain excluded through the project's clean repository structure and .gitignore rules.

10 | Testing & Validation Approach

Validation should be performed against authorized targets and controlled test environments. Component-level checks should be followed by end-to-end validation of the master workflow, with attention to successful/failed module states, consistency of collected observations, behavior-engine output, and the final unified score.

- Validate HTTP response capture, headers, cookies, redirects, and method checks.

- Validate security-header findings and severity classification.

- Validate passive user-discovery extraction and de-duplication behavior.

- Validate OSINT input handling and structured lookup responses.

- Validate robots.txt parsing and interesting-path severity classification.

- Validate behavior-engine anomaly/correlation output and score calculation.

- Validate unified scoring when one or more modules fail.

11 | Key Engineering Challenges

- Integrating multiple independent security modules into one coherent workflow.

- Reusing a shared network capture so behavioral analysis does not unnecessarily repeat collection.

- Turning low-level HTTP/network observations into structured behavioral evidence.

- Correlating indicators across multiple behavioral domains.

- Keeping reconnaissance, passive user discovery, and manual OSINT responsibilities distinct.

- Ensuring failed core modules are represented as failures rather than misleading security scores.

12 | Conclusion

The project demonstrates an integrated approach to web-security assessment by connecting reconnaissance, HTTP security analysis, behavioral profiling, and unified risk assessment. Its distinctive analytical layer is the Security Behavior Profiling Engine, which interprets patterns across collected observations and connects anomalies and correlated indicators to an overall assessment.

As a graduation project, the platform provides a practical demonstration of collaborative security engineering: separate technical modules are developed by different team members and then connected through a common assessment workflow and application interface.

13 | Future Enhancement Opportunities

- Expand the range and depth of security checks and reconnaissance modules.

- Increase behavioral-indicator coverage and correlation rules.

- Add richer dashboards and report visualizations.

Expand automated regression and integration testing.

Improve deployment configuration and production hardening.

Add configurable assessment profiles for different authorized testing scenarios.

14 | Acknowledgment

We are grateful to the NTI administration for providing the learning environment, resources, and opportunities that supported this practical experience. We also sincerely appreciate our instructor's guidance, constructive feedback, and technical mentorship throughout the project.

National Telecommunication Institute (NTI)

Appendix A | Repository Components

File	Purpose
app.py	Flask application entry point and API registration.
master_link.py	Master orchestration and unified scoring.
http_scanner.py	HTTP collection and security checks.
security_behavior_engine.py	Behavioral profiling and risk analysis.
user_discovery.py	Passive site-side identity discovery.
osint.py	OSINT CLI functionality.
osint_api.py	OSINT API lookup layer.
robots_analyzer.py	robots.txt parsing and path analysis.
index.html	Frontend structure.
script.js	Frontend behavior and API interaction.
style.css	Frontend styling.
README.md	Repository documentation.
.gitignore	Git exclusion rules for generated/local/sensitive files.